Initial Project Plan

Project Kickoff

Team Name: The Trailblazers

Sponsoring Organization: Adirondack Wilderness Advocates

Members/Roles:

- Nico Cabrera: Project Manager
- Ethan Patterson: Technical Lead
  General Security Lead
- Conlan Connor: UI/UX
  Team Website Maintainer
- Richard Tang: UI/UX
- Trevor Wiesen: AWS
  Cloud/API Security Lead
- Fabi Marrufo: AWS
  Coach/Sponsor/Engineering Communications
  Scribe

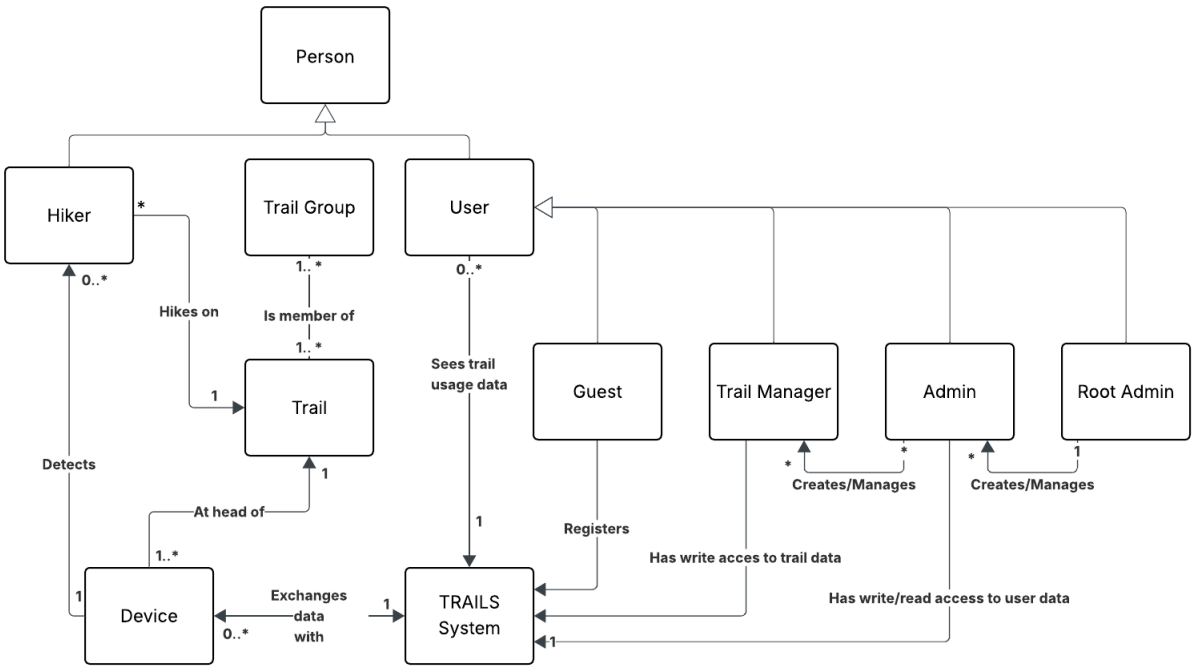
Project Synopsis

Hiking is a popular activity and the usage of hiking trails in the Adirondacks has been growing rapidly over recent years. Adirondack Wilderness Advocates is a wilderness advocacy organization that speaks for and protects the wilderness quality of the Adirondack Forest Preserve. There are over 900 different trailheads in the Adirondacks and collecting data from these remote areas to determine trail maintenance can be very time consuming and expensive. To solve this, a series of motion detector sensors are placed around the state park to detect hikers. Activity from the sensors is processed and displayed on a website via an easy to use interactive map of the park.

The implementation uses a cloud based software solution, building on a pre-existing architecture and framework created by previous project teams. This project aims to enhance the current prototype to turn it from a proof-of-concept into a professional software product.
The project will take an Agile approach by focusing on regular deliverables and continued sponsor engagement through demos and feedback. A “stretch goal” will be to develop this product to be mobile-friendly as well.

The team expects to utilize software engineering techniques/processes such as unit testing, DevOps and CI/CD, and also emphasize secure software development.

Domain Model



Development Methodology

The development methodology that the team and sponsor chose is Agile Scrum, with two week sprints and one sponsor meeting at the end of the sprint, on Wednesday. The exact meeting time and place will be communicated by the communications officer, Fabi Marrufo.

Justification

Description of Agile (Scrum)

The Agile (Scrum) Methodology is the practice of breaking a project down into many smaller components that can be implemented iteratively. This iterative approach takes place over small

sprints typically 1-2 weeks long, wherein project components or sprint items are taken from the backlog, assigned to developers, implemented, tested and reviewed, and deployed by the end of the sprint. At the end of the sprint, the sprint items that were completed are placed into the deployed bucket and the process starts again with the next sprint. Changes can be made to requirements and user stories in the backlog at any time, usually at the end of the sprint during a process called “backlog refinement.” The point of the methodology is to provide the customer with the maximum amount of value as quickly as possible, usually through the usage of minimum viable products (MVPs).

Pros of Agile

Some of the greatest advantages of Scrum and Agile is how quickly feature sets are produced and deployed. Agile allows new features to be available every sprint. The usage of fixed period sprints and backlog refinement also allows for backlog items to be changed every sprint. This allows the development team a great deal of flexibility sprint to sprint. Having a continuous deployment strategy and comprehensive testing plan that the code goes through every two weeks, deployments at the end of the sprint should produce high quality code that can be tested by users faster than other methodologies that only get features to the users relatively late in the project timeline. With deployments of features every two weeks, code can be used by customers for a much longer period. Due to this additional user testing, potential bugs, issues, and differences in expectation can be resolved much faster as well.

Cons of Agile

The advantages of Agile can also quickly become its disadvantages. Due to how easily the methodology handles changing requirements, it’s very easy to change requirements too much or add them beyond the original scope of the project. Without strong leadership, the development team can quickly lose focus on a well defined MVP. Additionally, quickly changing requirements and tight deadlines can create pressure to prioritize coding over documentation. Thus, even good documentation produced at the beginning of a project might be completely useless for future maintenance and extension if affected docs aren’t updated at the end of each sprint.

While it’s true that features get to users quicker, due to tight deadlines if all parties involved do not have similar levels of buy-in, the structure can quickly collapse. For instance, if a feature is delivered to a customer, but they do not actually use or test it until weeks later and find problems, going back to fix the problem would inevitably create thrashing for the development team.

Planned Activities and Artifacts

The project has many university-defined, sponsor-defined, and team-defined activities and artifacts to produce during the project.

Milestones

There are several major milestones that the team must achieve to be called successful by the end of the project. What these milestones are is of secondary importance compared to how these team plans achieve them. Following the Agile Scrum Methodology, the project will be broken down into a series of epics, features, and stories to be implemented during each sprint. These are defined in the team's work breakdown structures. With this information, the team will have an idea of how many user stories need to be implemented each sprint to achieve all of its goals by the end of the project.

Deployments

At the end of each sprint, there will be a meeting with the sponsor to present what was planned for the sprint, what was achieved, a presentation or demonstration of the features added, and a period for the sponsor to provide feedback and ask questions. This will take place on Wednesday, and the sponsor will have a few days to file issues in the Github Project and make the team aware of any issues that he's found after the meeting. This is so the team has time to perform backlog refinement at the end of the sprint and integrate any fixes into the plan for the next sprint.

Roles

The team has self-identified many roles internally important to their organization, and their are several roles defined by the university.

Major Roles

Some university-defined roles include the Project Manager, the Communications Officer, and the Technical Lead. The Project Manager, Nico Cabrera, is responsible for planning and coordinating the teams activities, managing deliverable deadlines, and maintaining the scope of the project and in the event that scope changes are required, makes sure that the team does not lose focus on the direction they are going or their MVP. Additionally he is responsible for the faithful execution of the project plan, tracking of measurements and metrics, managing the team's priorities, presenting the current project's health to the rest of the team, and deciding the artifacts that are needed for the team to best execute their own roles. The Communications Officer, Fabi Marrufo, is responsible for managing all incoming and outgoing communications between the team and any external stakeholders to include the sponsor, the coach, and engineering team. She is responsible for the drafting and approval of any communications the team wants to make to the aforementioned external stakeholders. She is responsible for making sure that any changes or important information communicated to her by the stakeholders is communicated effectively to the rest of the team. Additionally, she is responsible for effectively communicating any internal decisions or developments to stakeholders as it is relevant. The Technical Lead, Ethan Patterson, is responsible for understanding the entirety of the development process, environment, and implementation details. He provides technical advice and is the go-to representative of the team to elaborate on any technical details of the system. He is in charge of making sure that non-technical requirements are well translated into technical implementation details that make sense for the use case and follows good coding practices. He

is the software engineering subject matter expert among a team of dedicated software engineers.

There are also many internally-defined roles that the team has decided it needs. This includes Scribe and Team Website Maintainer. The Scribe, Fabi Marrufo, is responsible for taking notes during meetings in a way that is well organized and can adequately serve as a means to look back at past conversations, agreements, and changes to the project. The Team Website Maintainer, Conlan Connor, is responsible for making sure that any project artifacts produced by the team are effectively and cleanly communicated on the team's website.

Subteams

The team has decided that it needs two subteams to effectively function. One is the UI/UX subteam, formed by Richard Tang and Conlan Connor. The other team is the AWS subteam, formed by Trevor Wiesen, Fabi Marrufo, and Ethan Patterson. The UI/UX subteam is responsible for implementing all of the team's frontend changes, and the AWS subteam is responsible for implementing all changes to the AWS backend, databases, authentication, and testing pipeline. Trevor Wiesen is the AWS subteam lead, responsible for disseminating required work from the Project Manager to the rest of the subteam in a way that is most effective. He is also responsible for providing accurate reports of the subteams progress to the Project Manager. Richard Tang is the UI/UX subteam lead, with similar responsibilities.

Standards and Quality Practices

There are many practices that must be followed to ensure that code is of the utmost quality.

Standard Practices

Code reviews will be implemented to check the quality of unit and integration tests going through the testing pipeline. Without quality tests, developers can't know if their code will continue to work in adverse conditions. Additionally, a continuous integration (CI) and continuous deployment (CD) pipeline will be implemented via Github Actions that will allow developers to put their code through an AWS-like environment test. Afterwards, their code will be deployed to a user acceptance test environment where the sponsor can use the new features outside of the production environment. Once the sponsor has accepted the changes, they can be deployed to production.

Backups & Failure Recovery

The databases can be backed up in Amazon Web Services (AWS) where the code and databases are all hosted. This backup can be automatically used when encountering developer defined failure modes. A manual switch to back up is also essential to maintain the integrity of the system, as it's impossible for the developers to identify all possible failures the database can encounter.

Tools

The technical tools that the team will be using are most often a product of previous team's choices. However, tools for productivity, task tracking, artifact production, and others are up to the team.

Github Projects & Issues

Github Projects can be linked to repositories and their issue boards, making it easy to manage all the tasks that a repo will need. This way the sponsor can track code contributions as well as sprint progress from a single view, and the team can keep track of everything in a single place.

Deliverable Calendars

There is a major deliverable calendar that provides a visualization of the due dates of deliverables. This often helps team members manage their time more effectively. As new tasks are created, the deliverable calendar is also populated and used to track progress.

Sprint Planning

At the end of each sprint is a backlog refinement meeting, paired with the next's sprint planning. This sprint planning meeting is used to select which user stories will be included in the sprint, as well as planning poker to decide how many story points each user story will require. This will help the team track velocity as well as help with effort estimation.

Regular Meetings

Meeting with the team on a regular basis can help the team remain focused on their roles and their work. Standups would also help with this, however meeting too often can lead to meeting fatigue, which would be detrimental to the project.

Versioning

Versioning will be handled by Github. The team will branch off of the msd-dev branch with a trailblazers branch, and from there will create new branches for epics, and within those epic branches will have feature branches. At the end of the sprint when all testing is complete, the feature branches will be merged with the epic branch, retested, and then the epic branch merged with the trailblazers branch, and retested. Then the trailblazers branch will be opened to user acceptance testing.

CI/CD

Via Github Actions, a CI/CD pipeline will be established to automate the testing process and track metrics such as code coverage.

AWS

Amazon Web Services will host the website, the code, our authentication code, as well as the databases.

Measurements

The team will make many measurements during the project to compile into metrics it believes will assist the team's efforts to produce a meaningful MVP for the sponsor.

Estimated Effort

The estimated effort for a sprint will be measured in terms of story points. However many points that the team allotted for the current sprint is the estimated effort.

Actual Effort

The actual effort for a sprint will be measured in terms of story points, like estimated effort. However many points the team actually achieved in the sprint is the actual effort.

Tasks Planned

The number of tasks planned for the team is measured using meeting notes, the deliverable calendar, and Github Issues. The sum of these measurements constitutes that week's tasks planned, and should be tracked by each team member and reported to the Project Manager to be presented in the 4-Up.

Tasks Completed

The tasks actually completed should be a function of the tasks planned. However many tasks completed should be tracked by each team member and reported to the Project Manager. Each completed task should be clearly and easily explainable in a bullet point to be tracked in the metrics document.

Requirements

The number of requirements changed, added, and subtracted are measured by using sponsor and coach communications. Should a user story or feature definition change, this constitutes a requirements change. Should a new user story or feature need to be added or subtracted, this constitutes a requirement addition or subtraction.

Metrics

The metrics the team will use are a function of the measurements they took. They have been selected to give the idea of how far behind or ahead of schedule they are, how the scope is changing from sprint to sprint, and how accurate estimations in effort are.

Task Slippage

This is a function of tasks planned vs tasks completed. If tasks are not being completed that are planned for that sprint or week, then management knows that the team is either thrashing or being given too much work. Management can also see how the team is performing over long periods of time, and potentially foresee issues before they happen by diligently tracking changes in task slippage over time.

Requirements Volatility

Requirements addition, subtraction, and change should be tracked to give the team and team management an idea of how the scope of the project is changing from sprint to sprint, as scope creep is a known issue in Scrum. Should the aggregate scope change be over some threshold, then the team and team management will know that the team may start losing focus on their desired MVP, and team management should meet with the sponsor to realign on the scope of the project.

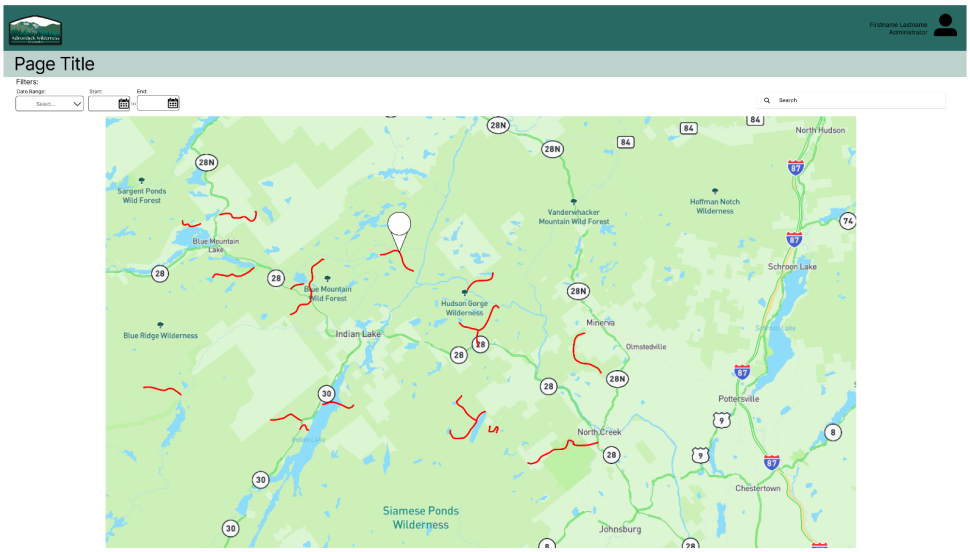
Effort Estimation Accuracy

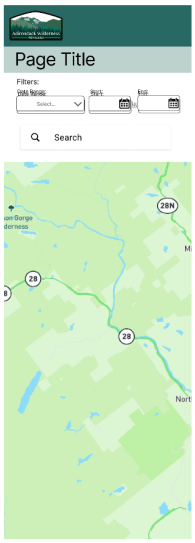
The accuracy of effort estimation is a function of the estimated effort vs the actual effort of a sprint. If these values differ greatly in any direction, then the team will know that they are either over or under estimating the difficulty of tasks, and can adjust accordingly.

Additional Elements

Flow and Use Cases

[/landing page]





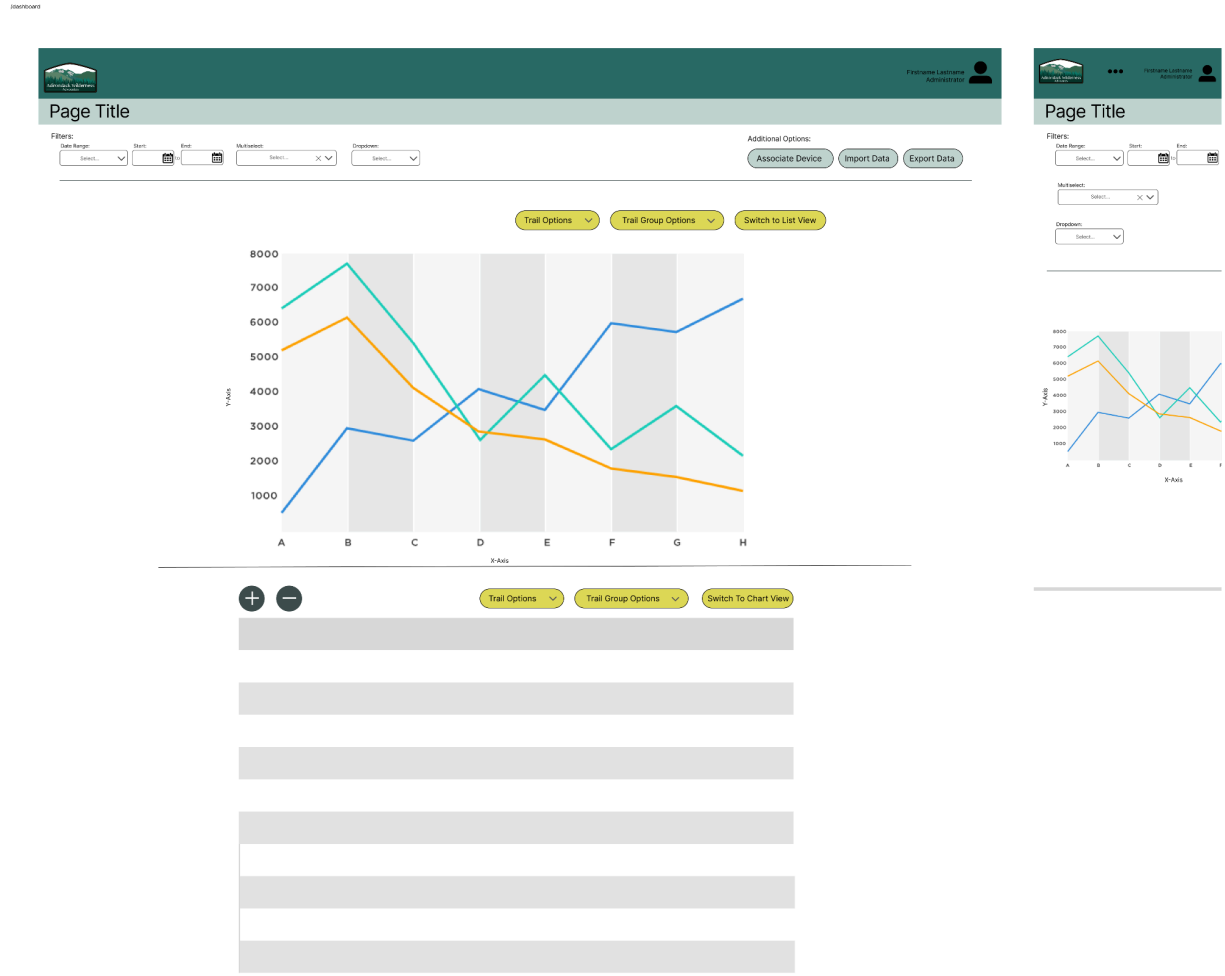
Not shown on this wire frame are the different colored pins at the trail heads.

Use Cases

1. Log in / Register (see /login)
    a. As users don't need to log in to see basic trail usage data, the landing page is the first thing they see.
2. Click on pin (see /dashboard)

    a. When a pin/trailhead is clicked on, it will route the user to the **dashboard view**, with a single trail selected in the trail filters. This will show the user the charted usage data over some default amount of time for the trail they selected.
3. Change timeframe of data
    a. The default is all data aggregated over all time periods, for which there is data.
    b. This should be a start/end date picker.
        i. Presets for the day before, last two weeks, last month, all time

[/dashboard]



Not shown here are the dropdown menus for trail and trail group CRUD, as well as the popups that would appear when clicked.

Use Cases:

1. Routed from landing page:

    a. Should show a filtered **chart view** for the trail the user selected on the landing page
    b. The **“default” filter** would be the trail the user selected on the landing page.
2. If a Trail Manager, Admin, or Root Admin:
    a. Should see all buttons
3. If a registered user:
    a. Should see a button for exporting trail data.
4. If an unregistered user (guest):
    a. Should only see the chart and filters, no buttons.
    b. If a guest wants to export data, they need to register (see /login).
        i. It should be clear through a greyed out button that exporting is possible, but if clicked a popup appears to tell the guest the above information.
5. Importing:
    a. Will display a file chooser requesting for the CSV file (path) from the user.
    b. Validates the structure and accepts or rejects the file depending on the results of validation.
    c. After validating the data, it should be uploaded to the persistence layer.
6. Exporting:
    a. Given the current filters applied for date range and trails, retrieves trail usage data from persistence layer.
    b. Will export the trail usage data to CSV format into users downloads folder.
7. Trail CRUD
    a. Must be able to create Trail
        i. Trail Name
        ii. Trail Location (trail head, should be lat/lon coordinates for map view)
    b. Must be able to update Trail
        i. Trail Name
        ii. Trail Location
    c. Must be able to delete Trail
8. Trail Group CRUD
    a. Must be able to create trail group
        i. Group name
        ii. Trails in group
            1. Should use a search tool rather than scroll view to add trails to the group
    b. Must be able to update trail group
        i. Group Name
        ii. Trails in group
            1. Same as above
    c. Must be able to delete trail group
9. Device CRUD
    a. Must be able to connect/”create” a device
        i. Manual "add device" function for adding offline devices to the persistence layer

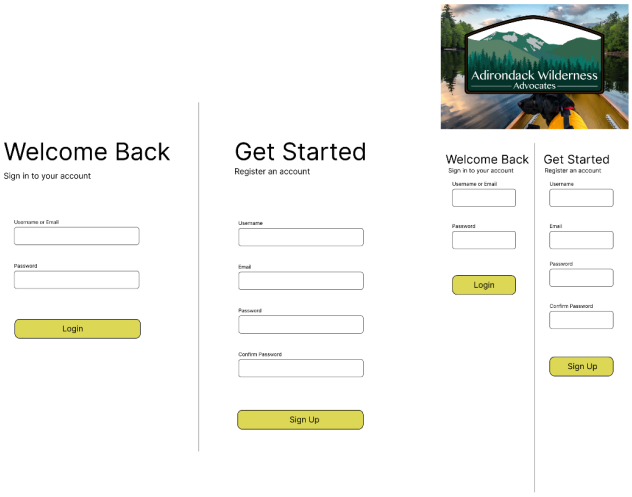
ii. Recorded in the persistence layer whether the device was added by API call (autonomously) or by a user (manually).

b. Must be able to update a device
   i. If a device is moved from one trail to another, the device must be associated with a new trail

c. Must be able to delete a device
   i. If a device is no longer being used, there must be a way to delete the device from the persistence layer.

[/login]

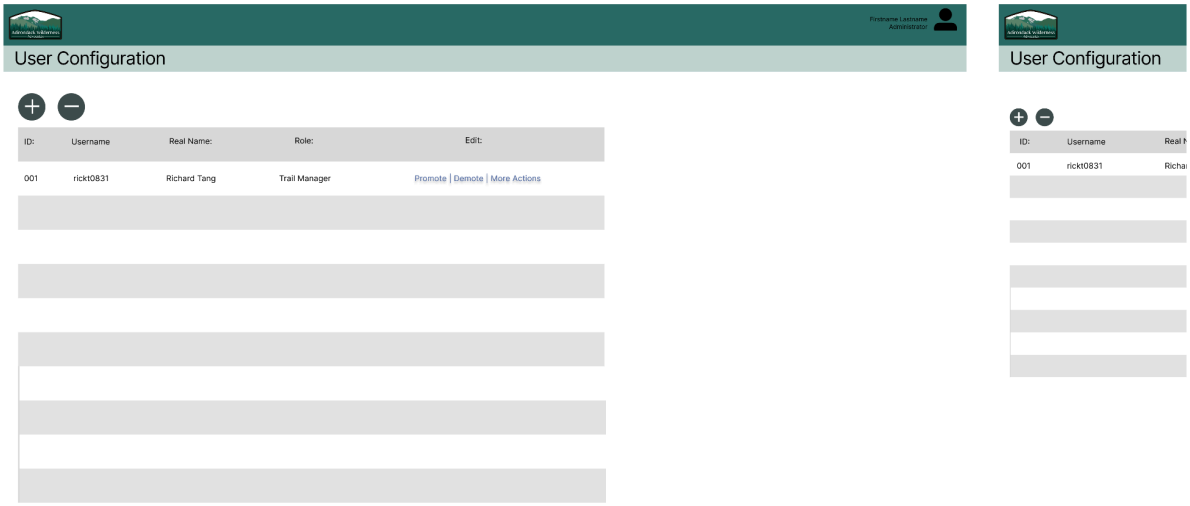
/login



Use Cases:

1. Register
   a. If you enter an unregistered email address AND username, you are able to register as a new user.
      i. If the email address or username has already been registered, it will be like you attempted to log in.
   b. If successful, the user will be redirected to the landing page.
2. Log In
   a. If you enter the correct username/email address and password, you will be logged into the system.
   b. If successful, the user will be redirected to the landing page.

[/privilege]



Use Cases:

1. Administrators and the Root Administrator can:
    a. Grant and remove trail manager permissions to registered users.
    b. Block/ban a certain user or trail manager.
2. The Root Administrator can:
    a. Grant and remove administrator permissions to registered users or trail managers.
    b. Block/ban a certain user, trail manager, or administrator.

Project Goals

The goals of the project are simple:

1. Convert the quality of the user experience from a student project to a professional product.
2. Implement rolling deployments to get code to the sponsor quickly for user testing.
3. Automate testing so that bugs that currently exist and might exist in the future are caught early.
4. Refactor the current code so that it is extensible for any future projects and maintainable by the sponsors organization.
5. Add user roles for better role-based access and management of the system.

Planned Milestones & Major Deliverables

There are a few major deliverables that are planned for the sponsor's usage. The biggest deliverables are a fully functional automated testing and deployment pipeline, as well as a refactored user interface that flows better and ultimately leads to a more harmonious user experience.

The automated testing and deployment pipeline is a major deliverable as it allows the team to safely and quickly test and deploy their code to the production environment without ever taking the site down. Outages were previously a major issue for the sponsor that gets solved here, as is a relatively empty looking site. Without many devices to transmit data to the system, it can be hard to test the system or show its capabilities to potential users and stakeholders. With an automated testing environment, specifically a user acceptance testing environment, the sponsor can populate data in the system and solve many of his problems at once.

The refactored user interface is important because as it stands, the project is not good quality enough to be presentable to any serious user or potential stakeholder. There are many UI glitches that undermine the trustworthiness of the system. By improving the user interface and experience, potential users and stakeholders are able to rebuild trust in the system and find it useful.

User Stories

Epic: Refactoring/Overhauling UI

- As a User, I want a user-friendly interface that seamlessly links relevant pages so I have a positive user experience.
- As a User, I want consistent designs across the application so that I can recognize different features no matter the page.
- As a User, I want a clean dashboard to view trail data, so that I can quickly see available features and pages of the application

Epic: Landing Page

- As a User, I want a landing page that displays an interactive trail map so that I can view trails and their location.
- As a Guest, I want to enter the page without logging in to see basic trail data

Epic: Account Registration/Login Page

- As a Guest, I want to be able to create an account so that I can access more details about trails
- As a User, I want to be able to login so I can access more details about trails

Epic: Privilege Management and Privilege-based Visibility

- As an Admin, I want to be able to see the privilege management page so that I can make necessary revisions to user roles
- As a Trail Manager, I want to be able to to see all of the necessary editing buttons when I am logged in so that I can properly make changes to aspects of the website
- As a Root Admin, I want to be able to create more admins on the privilege management page so that others can manage the system while I’m not present

- As a Root Admins, I want there to be admins and trail managers, so that certain people can have the power to do their specific job

Epic: Import/Export Function

- As a Trail Manger, I would like to be able to import data in the CSV format so that I can easily populate the website with historical trail usage data
- As a Trail Manager, I would like to be able to export trail usage data in the CSV format so that I may easily conduct offline analysis

Epic: Mobile Conversion

- As a User, I would like the webpage's formatting to adjust to my phone screen so that I am able to use this website on any device

Epic: Repository Updates

- As a product owner/developer(?) I want to have a clearly defined github repository structure so that I can work and manage the codebase without avoidable snags
- As a root admin(project owner?), I want a good security structure on the API/AWS so we don't have to update the keys on the devices if/when they're compromised
- As a project owner, I want the keys stored securely so we can open source the repository without worry of security compromises

Epic: CI/CD Testing

- As a product owner, I want regular and continuous testing on every code change so I can be sure that the website functionality is tested and validated before being released
- As a developer, I want to be able to push updates to the next stage automatically, so that I don't have to spend time manually doing it
- As a developer, I want to be able to host multiple environments simultaneously, so that I can use different environments for different tasks

Epic: Data Generation

- As a product owner, I want new simulated data appearing in the site, so that I can see and present what the site looks like without relying on real device input

Epic: Device Logging

- As a trail manager, I want to know when a device has an error and what the error is, so I can go out and service a broken/malfunctioning device when it needs it.

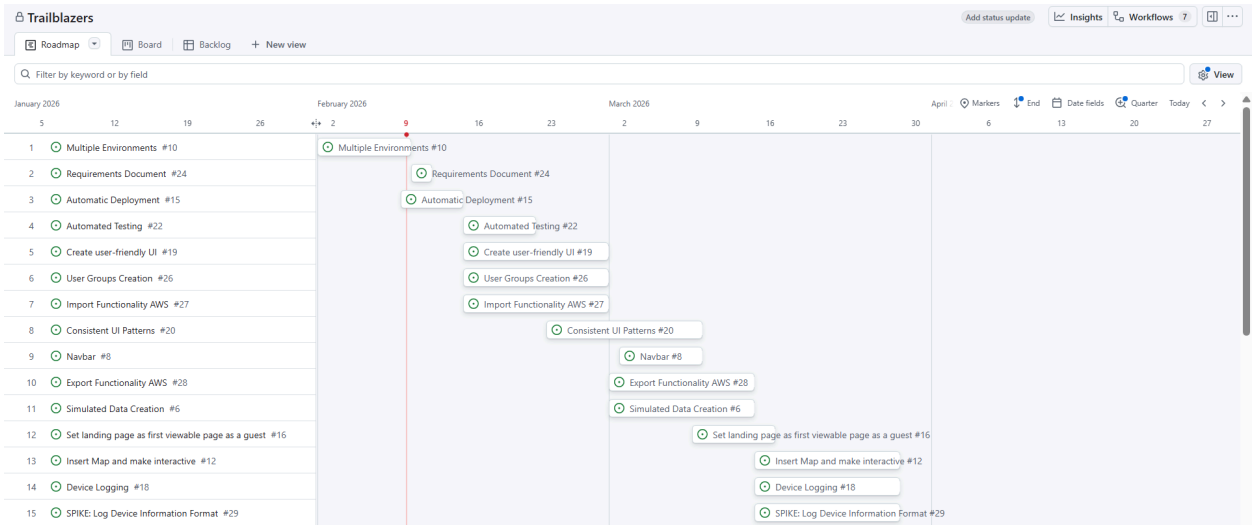
Epic: Automated Reporting

- As a user, I want to be able to see statistics on trails so I can make more informed decisions on what trails to use
- As a trail manager, I want to be able to see trail statistics so I can make more informed decisions on how to maintain and manage trails

Epic: URL Redirection

- As a user, I want to be able to go to “url here” (trails.adirondackwilderness.org?) to access the web application so that I can easily access the application
- As a developer/tester, I want to be able to go to a specific url to access a specific web application environment so that I can easily access the given environment

Work Breakdown Structure



Communication and stakeholder management plan

Stakeholder	Role	Level of Influence	Communication Requirements	Method	Frequency	Responsibility
Nico	PM	Low	Status reports	Discord	Daily	Team comm
Fabi	Dev	Low	Status reports	Discord	Daily	Sponsor + coach comm
Trevor	Dev	Low	Status reports	Discord	Daily	N/A
Richard	Dev	Low	Status reports	Discord	Daily	N/A
Conlan	Dev	Low	Status reports	Discord	Daily	N/A
Ethan	Dev	Low	Status reports	Discord	Daily	N/A
Mitra	Coach	Med	Approvals	Email	2-3 a week	Fabi comm
Craig	Sponsor	High	Approvals	Email	1 a week	Fabi comm

Risk Management Plan

See attached.